

Examen Partiel

Arbres binaires — Expressions arithmétiques — Tri par tas — ABR — LLRBT

 **Durée : 2h00**

 **Documents : interdits**

 **Calculatrice : interdite**

 **Barème : /40 pts**

Consignes générales :

- Répondre directement sur ce sujet. Soigner la présentation du code C (indentation, noms de variables).
- Toute fonction doit gérer les cas limites (arbre vide, tableau vide, pointeur NULL).
- Un résultat non justifié sera compté zéro même s'il est exact.
- Les 5 exercices sont indépendants et peuvent être traités dans n'importe quel ordre.

PARTIE A — THÉORIE (4 PTS)

- 1.a Donner la définition inductive d'un arbre binaire. Quelle est la relation entre le nombre 1 pt de nœuds internes n et le nombre de nœuds externes ?

- 1.b Pour un arbre binaire de taille $n \geq 1$, donner la hauteur **minimale** et la hauteur 1 pt **maximale**. Donner un exemple d'arbre atteignant chacune de ces bornes pour $n = 4$.

- 1.c Combien existe-t-il d'arbres binaires distincts de taille 3 ? Les dessiner *tous*. 1 pt

- 1.d Donner la récurrence satisfaite par les nombres de Catalan C_n . Calculer C_4 . 1 pt

PARTIE B — PROGRAMMATION C (4 PTS)

On utilise la structure : `typedef struct binary_tree { int label; struct binary_tree *left, *right; } *link;`

- 1.e Écrire la fonction récursive `int sum_binary_tree(link h)` qui renvoie la **somme** de 2 pts toutes les étiquettes de l'arbre (0 si vide).

```
int sum_binary_tree(link h) {  
  
  
  
  
  
  
}
```

- 1.f Écrire la fonction récursive `int height_binary_tree(link h)` qui renvoie la **hauteur** de 2 pts l'arbre (−1 si vide).

```
int height_binary_tree(link h) {  
  
  
  
  
  
  
}
```

Opérateurs : `+` `-` `*` `/` | Opérandes : chiffres `0-9` | Exemple : `+ * 2 3 4` = $(2 \times 3) + 4 = 10$

PARTIE A — COMPRÉHENSION (3 PTS)

- 2.a** Dessiner l'arbre correspondant à l'expression préfixée `* + 3 4 - 8 2`. Calculer sa valeur. **1 pt**

- 2.b** Écrire l'expression **préfixée** correspondant à l'arbre ci-dessous. Puis écrire son expression **infixe** (avec parenthèses). **1 pt**



- 2.c** Expliquer pourquoi `parse_aux` utilise un paramètre `const char**` plutôt que `const char*`. Quel problème surviendrait avec un simple `char*` ? **1 pt**

PARTIE B — PROGRAMMATION C (6 PTS)

- 2.d** Écrire `parse_aux` et `parse_expr` permettant de construire l'arbre à partir d'une chaîne en notation préfixée (espaces ignorés). **3 pts**

```
static link parse_aux(const char **s) {

}

link parse_expr(const char *s) {

}
```

- 2.e** Écrire `int eval_tree(link h)` qui évalue la valeur entière d'un arbre d'expression. **3 pts**

```
int eval_tree(link h) {

}
```

PARTIE A – THÉORIE (4 PTS)

3.a Quelle est la propriété d'un **tas-max** ? Donner les formules pour les fils gauche et droit d'un élément d'indice i dans un tableau base 1. **1 pt**

3.b Appliquer `buildHeap` sur le tableau suivant (montrer les étapes une à une) : 2 pts

Index : 1 2 3 4 5 6 7

4	10	3	5	1	2	6
---	----	---	---	---	---	---

3.c Quelle est la complexité de `buildHeap` ? Pourquoi est-ce $O(n)$ et non $O(n \log n)$? 1 pt

PARTIE B – PROGRAMMATION C (4 PTS)

Le tas est sur $a[l..r]$. Pour l'indice i : fils gauche = $l + 2*(i-l) + 1$, fils droit = $l + 2*(i-l) + 2$.

3.d Écrire `fixHeap(int *a, int l, int r, int i)` qui fait descendre `a[i]` à sa bonne place dans le tas `a[l..r]`. **4 pts**

```
void fixHeap(int *a, int l, int r, int i) {
```

}

Invariant ABR : clés gauche < racine $\leq$ clés droite. Fonctions disponibles : `size_BST`, `rotate_left`, `rotate_right`.

PARTIE A — THÉORIE (3 PTS)

- 4.a** Insérer dans l'ordre les valeurs **F, B, H, A, D, G, I** dans un ABR vide. Dessiner l'arbre obtenu. Donner son parcours infixe. **1 pt**

- 4.b** Sur l'ABR obtenu en 4.a, supprimer le nœud **F** (la racine) en utilisant `join_BST`. Décrire les étapes et dessiner l'arbre résultant. **1 pt**

- 4.c** Quel est le rôle de `partition_BST(h, k)` ? En quelle(s) brique(s) fondamentales se décompose-t-il ? Quelle est sa complexité ? **1 pt**

PARTIE B — PROGRAMMATION C (6 PTS)

- 4.d** Écrire `partition_BST(link h, int k)` qui amène le nœud de rang k à la racine (le rang est calculé via `size_BST(h->left)`). **3 pts**

```
link partition_BST(link h, int k) {  
  
  
  
  
  
  
  
  
  
}
```

- 4.e** Écrire `join_BST(link L, link R)` qui fusionne deux ABR avec $\max(L) < \min(R)$, **sans allouer de nouveau nœud**. **3 pts**

```
link join_BST(link L, link R) {  
  
  
  
  
  
  
  
  
  
}
```

PARTIE A — THÉORIE (3 PTS)

5.a

Énoncer les 5 propriétés d'un LLRBT. En déduire une borne supérieure sur sa hauteur.

2 pts

5.b

Quelle est la correspondance entre un LLRBT et un arbre 2-3 ? Comment un lien rouge encode-t-il un 3-nœud ?

1 pt

PARTIE B — SIMULATION (3 PTS)

5.c

Insérer les valeurs **S, E, A, R, C, H** (dans cet ordre) dans un LLRBT initialement vide. Dessiner l'arbre après chaque insertion, en indiquant les corrections appliquées (`rotate_left`, `rotate_right`, `color_flip`).

3 pts

Rappel : ordre des corrections = (1) `rotate_left` si rouge à droite → (2) `rotate_right` si double rouge à gauche → (3) `color_flip` si deux fils rouges. Racine toujours noire.

BARÈME RÉCAPITULATIF		
Exercice	Thème	Points
Exercice 1	Arbres binaires (théorie + code)	8
Exercice 2	Expressions arithmétiques préfixées	9
Exercice 3	Tri par tas (heapsort)	8
Exercice 4	Arbres Binaires de Recherche (ABR)	9
Exercice 5	LLRBT	6
TOTAL		40

```
/* — Pile (stack) — */
void  init_stack(struct stack *s, int max);
void  push(struct stack *s, link v);
link  pop(struct stack *s);
int   empty_stack(struct stack *s);

/* — File (queue) — */
void  init_queue(struct queue *q);
void  enqueue(struct queue *q, link v);
link  dequeue(struct queue *q);
int   empty_queue(struct queue *q);

/* — ABR utilitaires — */
int   size_BST(link h);           /* nombre de noeuds dans h */
link  rotate_left(link h);       /* monte le fils droit  - O(1) */
link  rotate_right(link h);      /* monte le fils gauche - O(1) */

/* — LLRBT — */
#define RED    1
#define BLACK  0
#define is_red(h)  ((h) != NULL && (h)->color == RED)
```